

TALLER PROGCOMP: TRACK STRINGS

FUNCIÓN Z

Gabriel Carmona Tabja

Universidad Técnica Federico Santa María,
Università di Pisa

April 29, 2025

Part I

FUNCIÓN Z

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(0) = 0$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(1) = 0$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(1) = 0$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(2) = 0$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(2) = 1$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(2) = 1$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(3) = 0$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(3) = 0$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(4) = 0$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(4) = 1$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(4) = 2$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(4) = 3$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(5) = 0$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacabaa

$$z(5) = 0$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$$z(6) = 0$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacabaa

$$z(6) = 1$$

FUNCIÓN Z

Definición

Dado un string s de largo n , se define la función siguiente:

$$z(i) = \max_{k=0 \dots n} \{k : s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Ejemplo

abacaba

$z(i)$	0	0	1	0	3	0	1
i	0	1	2	3	4	5	6

ALGORITMO TRIVIAL

```
1  vector<int> z_function_trivial(string s) {  
2      int n = s.size();  
3      vector<int> z(n);  
4      for (int i = 1; i < n; i++) {  
5          while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {  
6              z[i]++;  
7          }  
8      }  
9      return z;  
10 }
```

ALGORITMO TRIVIAL

```
1  vector<int> z_function_trivial(string s) {  
2      int n = s.size();  
3      vector<int> z(n);  
4      for (int i = 1; i < n; i++) {  
5          while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {  
6              z[i]++;  
7          }  
8      }  
9      return z;  
10 }
```

¿Cuál sería la complejidad?

ALGORITMO TRIVIAL

```
1  vector<int> z_function_trivial(string s) {  
2      int n = s.size();  
3      vector<int> z(n);  
4      for (int i = 1; i < n; i++) {  
5          while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {  
6              z[i]++;  
7          }  
8      }  
9      return z;  
10 }
```

¿Cuál sería la complejidad? $O(n^2)$

ALGORITMO TRIVIAL

```
1  vector<int> z_function_trivial(string s) {  
2      int n = s.size();  
3      vector<int> z(n);  
4      for (int i = 1; i < n; i++) {  
5          while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {  
6              z[i]++;  
7          }  
8      }  
9      return z;  
10 }
```

¿Cuál sería la complejidad? $O(n^2)$

¿Se podrá mejorar?

ALGORITMO TRIVIAL

```
1 vector<int> z_function_trivial(string s) {  
2     int n = s.size();  
3     vector<int> z(n);  
4     for (int i = 1; i < n; i++) {  
5         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {  
6             z[i]++;  
7         }  
8     }  
9     return z;  
10 }
```

¿Cuál sería la complejidad? $O(n^2)$

¿Se podrá mejorar? Sí :))

ALGORITMO EFICIENTE

Información previa

Se mantendrá una ventana $[l, r)$, tal que:

- ▶ r sea lo más a la derecha posible.
- ▶ $s[0 \dots r - l - 1] = s[l \dots r - 1]$

ALGORITMO EFICIENTE

Información previa

Se mantendrá una ventana $[l, r)$, tal que:

- ▶ r sea lo más a la derecha posible.
- ▶ $s[0 \dots r - l - 1] = s[l \dots r - 1]$

Caso 1: $i \geq r$

No tenemos información sobre $s[i \dots n - 1]$

1. Aplicamos algoritmo trivial
2. Actualizamos valores:
 - $l = i$
 - $r = i + z[i]$

ALGORITMO EFICIENTE

Información previa

Se mantendrá una ventana $[l, r)$, tal que:

- ▶ r sea lo más a la derecha posible.
- ▶ $s[0 \dots r - l - 1] = s[l \dots r - 1]$

Caso 2: $i < r$

Tenemos información sobre $s[i \dots r - 1]$

1. Inicializamos base $z[i] = \min(r - i, z[i - l])$
2. Aplicamos algoritmo trivial
3. Actualizamos l y r igual que en caso 1, si $i + z[i] > r$

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 1, l = 0, r = 0$$

$z(i)$	0	0	0	0	0	0	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 1, l = 1, r = 2$$

$z(i)$	0	1	0	0	0	0	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 2, l = 1, r = 2$$

$z(i)$	0	1	0	0	0	0	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 2, l = 1, r = 2$$

$z(i)$	0	1	0	0	0	0	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 3, l = 1, r = 2$$

$z(i)$	0	1	0	0	0	0	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 3, l = 3, r = 3$$

$z(i)$	0	1	0	0	0	0	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 4, l = 3, r = 3$$

$z(i)$	0	1	0	0	0	0	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 4, l = 4, r = 7$$

$z(i)$	0	1	0	0	3	0	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 5, l = 4, r = 7$$

$z(i)$	0	1	0	0	3	0	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 5, l = 4, r = 7$$

$z(i)$	0	1	0	0	3	$\min(2, z[1])$	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 5, l = 4, r = 7$$

$z(i)$	0	1	0	0	3	$\min(2, z[1])$	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 5, l = 4, r = 7$$

$z(i)$	0	1	0	0	3	1	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 5, l = 4, r = 7$$

$z(i)$	0	1	0	0	3	1	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 6, l = 4, r = 7$$

$z(i)$	0	1	0	0	3	1	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 6, l = 4, r = 7$$

$z(i)$	0	1	0	0	3	1	$\min(1, z[2])$
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$$i = 6, l = 4, r = 7$$

$z(i)$	0	1	0	0	3	1	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

Ejemplo

aabcaab

$z(i)$	0	1	0	0	3	1	0
i	0	1	2	3	4	5	6

ALGORITMO EFICIENTE

```
1
2 vector< int > zfunction(string &s) {
3
4     n = s.size();
5     vector< int > z(n, 0);
6     int l = 0, r = 0;
7     for (int i = 1; i < n; i++) {
8         if (i <= r)
9             z[i] = min(r - i + 1, z[i - l]);
10        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
11            ++z[i];
12        if (i + z[i] - 1 > r)
13            l = i, r = i + z[i] - 1;
14    }
15    return z;
16 }
```

ALGORITMO EFICIENTE

```
1
2 vector< int > zfunction(string &s) {
3
4     n = s.size();
5     vector< int > z(n, 0);
6     int l = 0, r = 0;
7     for (int i = 1; i < n; i++) {
8         if (i <= r)
9             z[i] = min(r - i + 1, z[i - l]);
10        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
11            ++z[i];
12        if (i + z[i] - 1 > r)
13            l = i, r = i + z[i] - 1;
14    }
15    return z;
16 }
```

La complejidad este algoritmo es $O(n)$.

ALGORITMO EFICIENTE

```
1
2 vector< int > zfunction(string &s) {
3
4     n = s.size();
5     vector< int > z(n, 0);
6     int l = 0, r = 0;
7     for (int i = 1; i < n; i++) {
8         if (i <= r)
9             z[i] = min(r - i + 1, z[i - l]);
10        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
11            ++z[i];
12        if (i + z[i] - 1 > r)
13            l = i, r = i + z[i] - 1;
14    }
15    return z;
16 }
```

La complejidad este algoritmo es $O(n)$.

Pueden encontrar una implementación en nuestro apunte [Handbook-USM](#)

Part II

RESOLVAMOS UN PROBLEMA :)

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- ▶ Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$
- ▶ Las primeras $n + 1$ posiciones corresponden al string s y $\$$.

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- ▶ Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$
- ▶ Las primeras $n + 1$ posiciones corresponden al string s y $\$$.
- ▶ Si $z[i] = n$ con $n < i < m + n + 1$, aparece s ! :)

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- ▶ Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$
 - ▶ Las primeras $n + 1$ posiciones corresponden al string s y $\$$.
 - ▶ Si $z[i] = n$ con $n < i < m + n + 1$, aparece s ! :)
- Extra:** Para encontrar la posición en t basta con hacer $i - n - 1$.

Ejemplo

$s = \text{hola}$

$t = \text{hola_estas..._hola?_chao}$

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- ▶ Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$
- ▶ Las primeras $n + 1$ posiciones corresponden al string s y $\$$.
- ▶ Si $z[i] = n$ con $n < i < m + n + 1$, aparece s ! :)

Extra: Para encontrar la posición en t basta con hacer $i - n - 1$.

Ejemplo

$s = \text{hola}$

$t = \text{hola_estas..._hola?_chao}$

$s + \$ + t = \text{hola\$hola_estas..._hola?_chao}$

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$
- Las primeras $n + 1$ posiciones corresponden al string s y $\$$.
- Si $z[i] = n$ con $n < i < m + n + 1$, aparece s ! :)

Extra: Para encontrar la posición en t basta con hacer $i - n - 1$.

Ejemplo

$s = \text{hola}$

$t = \text{hola_estas..._hola?_chao}$

$s + \$ + t = \text{hola$hola_estas..._hola?_chao}$

$s + \$ + t$	h	o	l	a	\$	h	o	l	a	_	e	s	t	a	s	.	.	.	_	h	o	l	a	?	_	c	h	a	o
z	0	0	0	0	0	4	0	0	0	0	0	0	0	0	0	0	0	0	4	0	0	0	0	0	0	1	0	0	

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$
- Las primeras $n + 1$ posiciones corresponden al string s y $\$$.
- Si $z[i] = n$ con $n < i < m + n + 1$, aparece s ! :)

Extra: Para encontrar la posición en t basta con hacer $i - n - 1$.

Ejemplo

$s = \text{hola}$

$t = \text{hola_estas..._hola?_chao}$

$s + \$ + t = \text{hola\$hola_estas..._hola?_chao}$

$s + \$ + t$	h	o	l	a	\$	h	o	l	a	_	e	s	t	a	s	.	.	.	_	h	o	l	a	?	_	c	h	a	o
z	0	0	0	0	0	4	0	0	0	0	0	0	0	0	0	0	0	0	0	4	0	0	0	0	0	0	1	0	0

REFERENCES I